Lab 2.1 — Reviewing a Requirements Document

Design of Software System · Turin Polytechnic University in Tashkent

Goal of this lab

In Lab 2 you wrote requirements yourself. In real work you will much more often receive a document written by somebody else and you will have to **review** it before the team starts to design and to code. A requirement that contradicts another one, or that nobody can test, is a defect — and a defect found in the requirements phase costs almost nothing, while the same defect found after release costs tens of times more. This lab is that review.

- There are **two exercises**. In each one you receive an extract of a requirements document (an SRS) that contains real mistakes, and you write a review report.
- No programming in this lab. Everything is reading, analysis and writing.
- Work alone. You may discuss the ideas with a classmate, but the text must be your own.

This lab is done during the lesson. Write your answers on paper or in your notebook; at the end of the class we go through them together. Nothing has to be sent anywhere — what counts is that you take part in the discussion.

The six kinds of defect you are looking for

Code	Kind of defect	How you recognise it
C	Contradiction	Two requirements cannot both be true at the same time.
A	Ambiguous	Two readers understand the sentence in two different ways.
T	Not testable	You cannot write a line that says how it would be checked; there is no number in it.
N	Not a requirement	It is a design decision, a business goal or a promise about the organisation, not about what the software must do.
W	Wrong type	A non-functional requirement written in the list of functional ones, or the opposite.
M	Missing	An obvious case that nobody wrote at all.

*Use exactly these letters in your tables. One line may have two letters (for example **T** and **W** together).*

Exercise 1 — The TTPU library system

A junior analyst gave you the extract below. It is the whole functional part of the SRS of the new library system of the university. Read it twice before you start writing.

```
FR-01  The system shall allow a registered student to borrow at most 5 books at a time.
FR-02  The system shall allow a student to borrow a 6th book if the librarian agrees.
FR-03  The loan period shall be 14 days for every book.
FR-04  Books marked "reference" shall never leave the library building.
FR-05  A student shall be able to renew a loan online for 14 more days.
FR-06  A reference book shall be renewed online in the same way as any other book.
FR-07  The system shall send an e-mail reminder 2 days before the due date.
FR-08  The system shall send the reminder only after the book is already late.
FR-09  The system must be user friendly.
FR-10  A student with 3 late returns shall be blocked from borrowing for 30 days.
FR-11  A blocked student shall still be able to borrow the books he needs for his thesis.
FR-12  The librarian shall be able to add, edit and delete a book record.
FR-13  The database shall be MySQL 8.
FR-14  The system shall be fast.
FR-15  Search results shall appear quickly.
FR-16  The system shall keep the full borrowing history of every student for ever.
FR-17  Personal data shall be deleted 1 year after the student leaves the university.
FR-18  The library shall buy 200 new books every year.
NFR-01 The system shall be available 99.5% of the working hours of the semester.
NFR-02 The system shall run on the two computers of the library reading room.
NFR-03 The system shall never fail.
```

1.1 Find the contradictions

There are **four** pairs of requirements that cannot both be true. Write them in a table: the two IDs, and one sentence saying why they cannot live together.

Pair	Why they contradict
FR-.. and FR-..	…

1.2 Find the lines that are not requirements at all

Three lines do not say what the software must do. For each one write the ID, what it really is (a design decision / a business goal / an impossible promise), and where it should be written instead (in the design document, in the project plan, or nowhere).

1.3 Find what cannot be tested and repair it

Four lines cannot be tested as they are written. For each one write the corrected requirement: it must contain a number and a unit, and one line that says how a tester would check it.

ID	Corrected requirement (with a number)	How it is tested
FR-14	The system shall show the result of a search of the whole catalogue in less than 2 seconds.	Search a title in a catalogue of 50 000 books and measure the time with a stopwatch.

1.4 Repair the contradictions

Take the four pairs of 1.1 and rewrite each pair so that both requirements can be implemented together. You are allowed to add a condition, an exception or a new number — but you are not allowed to simply delete one of the two. Write eight requirements in total.

Hint: most contradictions disappear when you write the missing condition. “A reference book shall not be renewed” is one line, and the document becomes correct.

Exercise 2 — The TTPU canteen pre-order system

The second document comes from another project. Here the analyst collected the sentences from three different people — a student, the cook and the manager — and wrote them into one list without reading them together. That is why the document looks reasonable line by line and is broken as a whole.

```
FR-01  The system shall let a student order a meal from the menu of the day.
FR-02  The student shall pay in the application before the order is sent to the kitchen.
FR-03  The student shall pay in cash at the counter when he takes the meal.
FR-04  The system shall accept orders between 08:00 and 11:00.
FR-05  The kitchen shall start cooking the orders at 11:30.
FR-06  A student shall be able to change his order until 11:45.
FR-07  The system shall show the number of portions left for every dish.
FR-08  A student shall be able to order a dish with 0 portions left; the kitchen cooks more.
FR-09  The system shall not accept an order for a dish with 0 portions left.
FR-10  Each order shall get a number that the student shows at the counter.
FR-11  The system shall send every order to the kitchen screen immediately.
FR-12  The system shall send the orders to the kitchen in one list at 11:00.
FR-13  The system shall work without the internet.
FR-14  The interface shall be in Uzbek, Russian and English.
FR-15  All messages of the system shall be in English.
FR-16  The manager shall see the money collected during the day at the end of the day.
FR-17  The system shall be written in Python with the Django framework.
FR-18  The canteen shall reduce the food it throws away by 30%.
FR-19  The system shall be secure.
FR-20  The system shall be ready before the beginning of the second semester.
NFR-01 The order screen shall open in less than 2 seconds on the university Wi-Fi.
NFR-02 The system shall serve 500 students ordering during the same hour.
NFR-03 The system shall be easy to use for a first-year student.
```

2.1 The full review table

Go through the 23 lines one by one and fill the table below. Write a row **only** for the lines that have a defect — there are **twelve** of them. Use the letters C, A, T, N, W from the first page.

ID	Defect	One sentence: what exactly is wrong	Corrected line (or “delete”)
FR-03	C	It contradicts FR-02: the payment cannot be both before and after the meal is prepared.	The student shall pay in the application; cash payment is not supported in version 1.

2.2 What nobody wrote

Five normal situations are missing from this document. Write them as five new requirements with IDs FR-21 … FR-25, each one with the word **shall** and a number where a number is needed.

- Think about: the student who paid and did not come; money back; a dish that the student cannot eat; days when the canteen is closed; who is allowed to open the manager screen.

2.3 The SDLC question — when should each defect have been found?

Choose **four** defects from your table of 2.1. For each one fill a row:

ID	In which phase is it found now?	In which phase should it have been found?	What happens if it is found only after release?
FR-12	Requirements review (now)	Requirements — the two sentences were written on the same day by two people.	The kitchen screen would have to be rewritten, and the cooks would already have learned the wrong way of working.

In the last column use the cost chart of Lecture 2: a change costs about **1** in the requirements phase, **5** in design, **10** during coding and **50–100** after release. Write the number you expect for your defect and one sentence explaining why.

2.4 One use case

Write the use case **“Student orders a meal”** in the form we used in the lecture: actor, precondition, main flow of 5–8 numbered steps, one alternative flow (the payment fails) and the postcondition. Your use case must respect the corrected requirements, not the broken ones.

Then add one line: which of the requirements FR-01 … FR-25 your use case covers. This line is the beginning of a traceability matrix.